

Contents

□ Emacs IDE - Roadmap de Melhorias Futuras	1
Sumário	1
1. Corporal Mode □	1
2. NNC - Neural Network Completion □	3
3. Memory Graph □	6
4. Temporal Navigation □	8
5. Context Navigation □	10
6. Context Management □	13
7. Context Buffers □	16
□ Instalação e Integração	19
□ Checklist de Implementação	20
□ Visão de Futuro	20

□ Emacs IDE - Roadmap de Melhorias Futuras

“A tecnologia é melhor quando conecta pessoas.” — Matt Mullenweg

Sumário

1. Corporal Mode □
 2. NNC - Neural Network Completion □
 3. Memory Graph □
 4. Temporal Navigation □
 5. Context Navigation □
 6. Context Management □
 7. Context Buffers □
-

1. Corporal Mode □

Definição

Corporal Mode é um sistema de feedback háptico e visual que traduz estados do Emacs (compilação, lint, sugestões LSP) em sinais físicos e visuais tangíveis.

Filosofema

“O corpo é a primeira interface. Antes de ler, sentimos; antes de entender, percebemos.”

Rationale: A maioria dos desenvolvedores depende exclusivamente de feedback visual. O Corporal Mode adiciona uma camada de percepção corporal (vibração, sons

contextuais, luzes RGB) que permite reagir a eventos do ambiente de desenvolvimento sem precisar olhar para a tela constantemente.

Componentes Técnicos

```
;; ~/.emacs.d/lisp/corporal.el

(defvar corporal-haptic-device nil
  "Dispositivo háptico conectado (e.g., 'keyboard-led', 'controller').")

(defvar corporal-event-sounds
  '(("compile-success" . "~/sounds/bell.wav")
    ("compile-error" . "~/sounds/error.wav")
    ("lsp-warning" . "~/sounds/warning.wav")
    ("lsp-info" . "~/sounds/info.wav"))
  "Mapeamento de eventos para sons.")

(defun corporal-trigger-feedback (event-type)
  "Dispara feedback para EVENT-TYPE."
  (pcase event-type
    ('compile-success
     (corporal-play-sound "compile-success")
     (corporal-flash-led "green" 200))
    ('compile-error
     (corporal-play-sound "compile-error")
     (corporal-flash-led "red" 500)
     (corporal-vibrate-keyboard 100))
    ('lsp-warning
     (corporal-play-sound "lsp-warning")
     (corporal-flash-led "yellow" 300))
    ('lsp-info
     (corporal-play-sound "lsp-info"))))

(defun corporal-flash-led (color duration-ms)
  "Alterna cor do LED do teclado por DURATION-MS ms."
  (when (executable-find "ghid")
    (start-process "corporal-led" nil "ghid" "led" color)))

(defun corporal-vibrate-keyboard (intensity)
  "Vibra o teclado com INTENSITY (0-100)."
  (when (executable-find "xhkd")
    (start-process "corporal-vib" nil "xhkd" "vibrate"
      (number-to-string intensity))))

;; Hooks de integração
(add-hook 'compilation-finish-functions
  (lambda (_buf _status)
    (if (string-match "finished" compilation-last-check)
```

```

      (corporal-trigger-feedback 'compile-success)
      (corporal-trigger-feedback 'compile-error))))

(add-hook 'flycheck-after-syntax-check-hook
  (lambda ()
    (pcase flycheck-last-status-change
      ('finished
        (if (eq flycheck-current-errors nil)
            (corporal-trigger-feedback 'compile-success)
            (corporal-trigger-feedback 'compile-error))))))

```

ELI5

Imagine que você está jogando videogame. Quando você acerta, o controle vibra e aparece uma luz verde. O Corporal Mode faz a mesma coisa, mas para programar: se o código compilar certinho, o teclado pisca verde e toca um som feliz. Se der erro, ele vibra e pisca vermelho para você já saber que precisa consertar algo.

Integração com Sistema

- **Compilação:** Feedback instantâneo ao terminar (M-x compile)
- **Flycheck/LSP:** Alerta sobre erros sem precisar olhar o buffer
- **Git:** Notificação háptica ao fazer commit (stash, push, pull)
- **Timer:** Vibração suave a cada Pomodoro (25 min)

2. NNC - Neural Network Completion ☐

Definição

NNC é um sistema de auto-completar inteligente que usa modelos neurais locais (LLM) para oferecer sugestões de código contextuais, ☐☐☐ o autocomplete tradicional.

Filosofema

“A inteligência não é prever o próximo passo, mas entender o caminho.”

Rationale: O autocomplete padrão usa completamento por prefixo (abbrev tables) ou análise sintática (company-capf). O NNC vai além, usando um modelo neural local (Mistral, Llama, Phi) para gerar sugestões semanticamente relevantes baseadas no contexto do projeto.

Arquitetura

```
;; ~/.emacs.d/lisp/nnc.el
```

```

(defvar nnc-model-path (expand-file-name "~/local/share/nnc/models/mistral-7b.Q4_1"
  "Caminho para o modelo GGUF local.")

(defvar nnc-server-process nil
  "Processo do servidor NNC.")

(defvar nnc-context-window 2048
  "Tokens de contexto para o modelo.")

(defvar nnc-suggestions '()
  "Lista de sugestões atuais.")

(defun nnc-start-server ()
  "Inicia o servidor NNC com llama.cpp."
  (interactive)
  (let ((server-cmd (list "llama-server"
    "-m" nnc-model-path
    "-c" (number-to-string nnc-context-window)
    "--host" "127.0.0.1"
    "--port" "8080"
    "-ngl" "99"))))
    (setq nnc-server-process
      (make-process
        :name "nnc-server"
        :command server-cmd
        :filter #'nnc-server-filter
        :sentinel #'nnc-server-sentinel))
    (message "▢ NNC: Servidor iniciado na porta 8080")))

(defun nnc-server-filter (proc output)
  "Filtra saída do servidor NNC."
  (when (buffer-live-p (process-buffer proc))
    (with-current-buffer (process-buffer proc)
      (insert output)))))

(defun nnc-server-sentinel (proc event)
  "Sentinela para eventos do servidor."
  (pcase event
    ("exited\n" (message "△ NNC: Servidor encerrado"))
    ("deleted\n" (message "▢ NNC: Servidor removido"))))

(defun nnc-get-completion (prompt callback)
  "Solicita completion para PROMPT e executa CALLBACK com resultado."
  (url-retrieve
    (format "http://127.0.0.1:8080/completion")
    (lambda (status)
      (goto-char (point-min))
      (re-search-forward "^$")

```

```

        (let ((response (json-parse-string (buffer-substring (point) (point-max)))))
          (funcall callback (alist-get 'content response)))))

(defun nnc-complete-at-point ()
  "Hook para company-mode: oferece completions neurais."
  (when (and (bound-and-true-p nnc-server-process)
              (process-live-p nnc-server-process))
    (let* ((context (nnc-get-buffer-context))
           (prompt (nnc-build-prompt context))
           (nnc-get-completion
            prompt
            (lambda (completion)
              (setq nnc-suggestions (list completion))))))
      '((" Suggestion" :annotation "Neural completion"))))

(defun nnc-get-buffer-context ()
  "Extraí contexto relevante do buffer atual."
  (let ((start (max (point-min) (- (point) 2000)))
        (end (min (point-max) (+ (point) 500))))
    (buffer-substring-no-properties start end)))

(defun nnc-build-prompt (context)
  "Constrói prompt para o modelo com CONTEXT0."
  (format "Você é um programador especialista. Complete o código:\n\n%s"
          context))

;; Integração com company
(add-to-list 'company-backends '(company-nnc :separate company-capf))

```

ELI5

Lembra quando você está escrevendo uma frase no celular e ele sugere a próxima palavra? O NNC faz isso, mas muito mais esperto. Em vez de só completar “fun” para “function”, ele entende que você está fazendo uma função que soma números e sugere: “return a + b; }”

Recursos

- **Contexto de projeto:** Analisa arquivos relacionados
- **Estilo consistente:** Aprende o estilo do seu código
- **Multilingual:** Suporte a qualquer linguagem
- **Offline:** Não precisa de internet
- **Privado:** Tudo fica na sua máquina

3. Memory Graph

Definição

Memory Graph é uma visualização interativa da estrutura de memória e estado do projeto, mostrando conexões entre funções, variáveis, imports e dependências em tempo real.

Filosofema

"A memória não é um arquivo, é uma teia. Cada conexão conta uma história."

Rationale: À medida que projetos crescem, a complexidade aumenta exponencialmente. O Memory Graph oferece uma visão holográfica do código, permitindo identificar dependências, dívida técnica e áreas de risco antes que se tornem problemas.

Implementação

```
;; ~/.emacs.d/lisp/memory-graph.el

(defvar memory-graph-buffer "*Memory Graph*"
  "Buffer para o grafo de memória.")

(defvar memory-graph-data '()
  "Dados do grafo extraídos do projeto.")

(defvar memory-graph-process nil
  "Processo do visualizador de grafo.")

(defun memory-graph-generate ()
  "Gera grafo de memória do projeto atual."
  (interactive)
  (setq memory-graph-data (memory-graph-extract-project))
  (memory-graph-render))

(defun memory-graph-extract-project ()
  "Extrai estrutura do projeto atual."
  (let ((files (project-files (project-root)))
        (nodes '())
        (edges '()))
    (dolist (file files)
      (when (member (file-name-extension file) '("el" "py" "js" "ts"))
        (let ((defs (memory-graph-find-definitions file)))
          (push (cons file defs) nodes)
          (setq edges (append edges (memory-graph-find-refs file defs))))))
    `(:nodes ,nodes :edges ,edges)))

(defun memory-graph-find-definitions (file)
```

```

"Encontra definições (funções, classes, variáveis) em FILE."
(with-temp-buffer
  (insert-file-contents file)
  (let ((defs '()))
    (goto-char (point-min))
    (while (re-search-forward
      (concat "\\(?:defun\\|defvar\\|defclass\\|"
        "function\\s +\\w+\\|class\\s +\\w+\\)"
        "\\s +\\(\\w+\\)")
      nil t)
      (push (match-string 1) defs))
    defs)))

(defun memory-graph-find-refs (file defs)
  "Encontra referências às definições em FILE."
  (with-temp-buffer
    (insert-file-contents file)
    (let ((refs '()))
      (dolist (def defs)
        (goto-char (point-min))
        (while (re-search-forward (regexp-quote def) nil t)
          (push (list def (count-lines 1 (point))) refs)))
      refs)))

(defun memory-graph-render ()
  "Renderiza o grafo usando graphviz ou d3."
  (let ((dot-file (expand-file-name ".memory-graph.dot" (project-root))))
    (with-temp-file dot-file
      (insert "digraph MemoryGraph {\n")
      (insert "  rankdir=LR;\n")
      (insert "  node [shape=box style=filled];\n")
      (dolist (node memory-graph-data)
        (pcase-let ((`(_ (:nodes ,nodes) (:edges ,edges))) node)
          (dolist (n nodes)
            (insert (format "  \"%s\" [fillcolor=lightblue];\n" (car n))))
          (dolist (e edges)
            (insert (format "  \"%s\" -> \"%s\";\n" (car e) (cadr e)))))
        (insert "}\n"))
      (start-process "graphviz" nil "dot" "-Tsvg" dot-file "-o"
        (concat dot-file ".svg"))
      (browse-url (concat "file://" dot-file ".svg"))))

;; Comando interativo
(global-set-key (kbd "C-c M-g") #'memory-graph-generate)

```

ELI5

Pense em um mapa do metrô. Cada estação é uma função do seu código, e os trilhos são as conexões entre elas. O Memory Graph é como ter um mapa completo do seu código, mostrando quais funções chamam quais, onde estão os engarrafamentos (funções muito chamadas) e as estações abandonadas (código morto).

Recursos

- **Visualização em tempo real:** Atualiza conforme você edita
 - **Zoom semântico:** Navegue entre níveis de abstração
 - **Heatmap:** Áreas de alta complexidade visualizadas
 - **Deteção de código morto:** Funções nunca chamadas
 - **Impact analysis:** O que quebra se eu mudar X?
-

4. Temporal Navigation □

Definição

Temporal Navigation permite navegar pelo histórico temporal de um arquivo ou projeto, mostrando não apenas o que mudou, mas quando e por quê.

Filosofema

“O tempo não é uma linha, é uma espiral. Voltamos sempre ao mesmo tema, com maior compreensão.”

Rationale: O Git mostra changesets, mas não captura a intenção ou o contexto temporal. Temporal Navigation adiciona camadas de tempo, permitindo ver a evolução do código como uma narrativa, não apenas como diffs.

Implementação

```
;; ~/.emacs.d/lisp/temporal-nav.el

(defvar temporal-nav-buffer "*Temporal Navigation*"
  "Buffer para navegação temporal.")

(defvar temporal-nav-gitlog nil
  "Cache do git log atual.")

(defvar temporal-nav-current-time nil
  "Tempo atual no slider.")

(defun temporal-nav-open ()
  "Abre painel de navegação temporal."
```



```

(interactive)
(setq temporal-nav-gitlog (temporal-nav-get-gitlog))
(pop-to-buffer temporal-nav-buffer)
(temporal-nav-render))

(defun temporal-nav-get-gitlog ()
  "Extraí git log formatado com datas e mensagens."
  (let ((log-output
        (shell-command-to-string
         "git log --format='%H|%ad|%an|%s' --date=iso")))
    (mapcar (lambda (line)
              (let ((parts (split-string line "|")))
                (list :hash (car parts)
                      :date (cadr parts)
                      :author (caddr parts)
                      :message (caddr parts))))
            (split-string log-output "\n" t))))

(defun temporal-nav-render ()
  "Renderiza a interface de navegação temporal."
  (let ((inhibit-read-only t))
    (erase-buffer)
    (insert "□ NAVEGAÇÃO TEMPORAL\n")
    (insert "===== \n\n")
    (insert "[Slider de Tempo] ←————→ \n\n")
    (insert "Selecione um ponto no tempo: \n\n")
    (dolist (entry temporal-nav-gitlog)
      (pcase-let ((`(:hash ,hash :date ,date :author ,author :message ,msg) entry))
        (insert (format "[%s] %s\n" date msg))
        (insert (format "  □ %s | □ %s\n" author (substring hash 0 7)))))
    (use-local-map (copy-keymap special-mode-map))
    (local-set-key (kbd "RET") #'temporal-nav-view-commit)
    (local-set-key (kbd "n") #'temporal-nav-next)
    (local-set-key (kbd "p") #'temporal-nav-prev))

(defun temporal-nav-view-commit ()
  "Mostra commit selecionado."
  (beginning-of-line)
  (re-search-forward "\\([a-f0-9]\\{7\\}\\)")
  (let ((hash (match-string 1)))
    (magit-show-commit hash)))

(defun temporal-nav-next ()
  "Vai para próximo commit no tempo."
  (interactive)
  (forward-line 3))

(defun temporal-nav-prev ()

```

```

"Vai para commit anterior no tempo."
(interactive)
(let ((lines-to-move (if (bobp) 0 3)))
  (forward-line (- lines-to-move)))

(defun temporal-nav-timemachine ()
  "Entra no modo viagem no tempo do git."
  (interactive)
  (let ((hash (completing-read "Commit: "
                              (mapcar (lambda (e) (alist-get :hash e))
                                      temporal-nav-gitlog))))
    (magit-checkout hash)))

;; Integração com which-key
(push '("temporal" . (("t" . "timemachine") ("n" . "nav"))) which-key-replacement-)

(global-set-key (kbd "C-c T") #'temporal-nav-open)

```

ELI5

Lembra do DeLorean do De Volta para o Futuro? O Temporal Navigation é como ter um DeLorean para o seu código. Você pode voltar no tempo e ver como o código era antes, ou avançar para ver o futuro planejado. É como ter um gravador de vídeo para cada momento da vida do seu projeto.

Recursos

- **Timeline visual:** Slider interativo de tempo
- **Blame temporal:** Quem mudou, quando e por quê
- **Time-lapse:** Watch mode que reproduz commits em sequência
- **Anotações:** Adicione notas explicativas a qualquer ponto
- **Bookmarking:** Marque momentos importantes

5. Context Navigation

Definição

Context Navigation permite navegar entre contextos de edição (funções, blocos, seções) usando atalhos contextuais que mudam baseado no modo maior atual.

Filosofema

“Navegar é escolher. Cada contexto é um universo; cada atalho, um portal.”

Rationale: A navegação tradicional usa M-g M-g (goto-line) ou isearch. Context Navigation vai além, entendendo a estrutura semântica do código e oferecendo navegação inteligente baseada no contexto.

Implementação

```
;; ~/.emacs.d/lisp/context-nav.el

(defvar context-nav-current-context nil
  "Contexto atual detectado.")

(defvar context-nav-jump-table
  '((emacs-lisp-mode
    ("f" . context-nav-lisp-defun)      ; Função
    ("v" . context-nav-lisp-variable)  ; Variável
    ("c" . context-nav-lisp-macro))    ; Macro
    (python-mode
    ("f" . context-nav-python-defun)    ; Def/class
    ("a" . context-nav-python-argument) ; Argumento
    ("i" . context-nav-python-import)) ; Import
    (js2-mode
    ("f" . context-nav-js-function)
    ("c" . context-nav-js-class)
    ("m" . context-nav-js-method))
    (org-mode
    ("h" . context-nav-org-heading)
    ("t" . context-nav-org-todo)
    ("c" . context-nav-org-checkbox)))
  "Tabela de jump contexts por modo.")

(defun context-nav-get-context ()
  "Detecta o contexto atual do ponto."
  (let ((contexts '()))
    (cond
      ((region-active-p) (push "region" contexts))
      ((and (fboundp 'treesit-node-at-point)
            (treesit-node-at-point))
       (push "tree-sitter" contexts))
      ((or (looking-at "^(") (looking-back "^("))
       (push "lisp-sexp" contexts)))
    contexts))

(defun context-nav-jump ()
  "Jump interativo usando contexto atual."
  (interactive)
  (let* ((mode major-mode)
         (jumps (or (alist-get mode context-nav-jump-table)
                    (alist-get 'fundamental-mode context-nav-jump-table)))
         (key (read-key-sequence "Jump: ")))
    (let ((cmd (lookup-key (make-sparse-keymap) key)))
      (when-let ((action (cdr (assoc (string key) jumps))))
        (call-interactively action))))))
```

```
;; Implementações específicas por modo
(defun context-nav-lisp-defun ()
  "Pula para o defun atual."
  (interactive)
  (beginning-of-defun)
  (re-search-forward "defun\\s +\\((\\w+\\)")
  (message "  Função: %s" (match-string 1)))

(defun context-nav-lisp-variable ()
  "Pula para a variável atual."
  (interactive)
  (let ((defs '()))
    (save-excursion
      (goto-char (point-min))
      (while (re-search-forward "defvar\\s +\\((\\w+\\)" nil t)
        (push (match-string 1) defs)))
    (message "Variáveis: %s" (mapconcat 'identity defs ", "))))

(defun context-nav-python-defun ()
  "Pula para def/class atual."
  (interactive)
  (re-search-backward "^\\s *\\(def\\|class\\)")
  (forward-word 2)
  (message "  %s" (current-word)))

(defun context-nav-org-heading ()
  "Pula para o próximo título org."
  (interactive)
  (org-next-visible-heading 1)
  (message "  %s" (org-get-heading)))

;; Keybindings
(global-set-key (kbd "M-g M-j") #'context-nav-jump)
(define-key goto-keymap (kbd "j") #'context-nav-jump)
```

ELI5

É como um GPS para o seu código. Quando você está em uma cidade, o GPS mostra ruas. Quando está em casa, mostra cômodos. O Context Navigation entende onde você está (função, classe, título) e oferece atalhos que fazem sentido para aquele lugar.

Recursos

- **Smart jumping:** Adapta-se ao modo atual
- **History:** Mantém stack de contextos visitados

- **Breadcrumbs:** Mostra caminho de navegação
 - **Marks:** Marque pontos para voltar depois
 - **Helm integration:** Integração com helm para busca avançada
-

6. Context Management

Definição

Context Management gerencia múltiplos contextos de trabalho simultâneos, permitindo switch rápido entre diferentes tarefas, projetos e mentalidades.

Filosofema

"O contexto é a moldura; a gestão é a arte de saber quando trocar de quadro."

Rationale: Desenvolvedores frequentemente trabalham em múltiplos projetos ou features simultaneamente. Context Management oferece workspaces lógicos que preservam estado completo (buffers, janelas, variáveis) e permitem switching instantâneo.

Implementação

```
;; ~/.emacs.d/lisp/context-mgmt.el
```

```
(defvar context-list '()
  "Lista de contextos salvos.")
```

```
(defvar current-context nil
  "Contexto ativo atual.")
```

```
(defvar context-ring-size 10
  "Número máximo de contextos no ring.")
```

```
(cl-defstruct context
  name                ; Nome do contexto
  buffers              ; Buffers abertos
  window-config        ; Configuração de janelas
  point-alist          ; Posição do ponto por buffer
  variables             ; Variáveis dinâmicas
  created              ; Timestamp de criação
  tags)                ; Tags para categorização
```

```
(defun context-save (&optional name)
  "Salva o contexto atual com NAME."
  (interactive (list (read-string "Nome do contexto: ")))
  (let ((ctx (make-context
                  :name (or name (format "context-%s" (format-time-string "%Y%m%d-%H%M"))
```

```

        :buffers (mapcar #'buffer-name (buffer-list))
        :window-config (current-window-configuration)
        :point-alist (context-capture-points)
        :variables (context-capture-vars)
        :created (current-time)
        :tags (context-detect-tags))))
(push ctx context-list)
(message "□ Contexto '%s' salvo!" (context-name ctx)))

(defun context-capture-points ()
  "Captura posição do ponto em todos os buffers."
  (mapcar (lambda (buf)
            (cons (buffer-name buf)
                  (with-current-buffer buf (point))))
          (buffer-list)))

(defun context-capture-vars ()
  "Captura variáveis dinâmicas importantes."
  `((fill-column . ,fill-column)
    (truncate-lines . ,truncate-lines)
    (current-prefix-arg . ,current-prefix-arg)
    (last-command . ,last-command)))

(defun context-switch (name)
  "Muda para o contexto NAME."
  (interactive
   (list (completing-read "Contexto: "
                          (mapcar #'context-name context-list))))
  (let ((ctx (cl-find name context-list
                     :key #'context-name :test #'string=)))
    (unless ctx (error "Contexto '%s' não encontrado" name))
    (context-restore ctx)))

(defun context-restore (ctx)
  "Restaura o contexto CTX."
  (when current-context
    (context-save (context-name current-context)))
  (set-window-configuration (context-window-config ctx))
  (dolist (pair (context-point-alist ctx))
    (let ((buf (get-buffer (car pair))))
      (when buf
        (with-current-buffer buf
          (goto-char (cdr pair))))))
  (dolist (var (context-variables ctx))
    (set (car var) (cdr var)))
  (setq current-context ctx)
  (message "□ Restaurado: %s" (context-name ctx)))

```

```

(defun context-detect-tags ()
  "Detecta tags baseadas no conteúdo."
  (let ((tags '()))
    (dolist (buf (buffer-list))
      (with-current-buffer buf
        (when (eq major-mode 'org-mode)
          (push "org" tags))
        (when (eq major-mode 'magit-status-mode)
          (push "git" tags))))
    (delete-dups tags)))

;; Comandos interativos
(global-set-key (kbd "C-c M-s") #'context-save)
(global-set-key (kbd "C-c M-r") #'context-switch)

(defun context-list-all ()
  "Lista todos os contextos."
  (interactive)
  (with-output-to-temp-buffer "*Contexts*"
    (dolist (ctx context-list)
      (princ (format "[%s] %s | Tags: %s\n"
                    (format-time-string "%Y-%m-%d" (context-created ctx))
                    (context-name ctx)
                    (mapconcat 'identity (context-tags ctx) ", "))))))

(global-set-key (kbd "C-c M-l") #'context-list-all)

```

ELI5

É como ter várias mesas de trabalho. Você pode estar programando em uma mesa, escrevendo documentos em outra, e verificando emails em outra. Cada mesa tem suas coisas espalhadas do jeito que você deixou. Context Management faz o mesmo, mas para o Emacs.

Recursos

- **Named contexts:** Nomeie contextos por projeto/tarefa
 - **Auto-save:** Salva contexto automaticamente ao sair
 - **Persistente:** Salva contextos entre sessões
 - **Tags:** Categorize contextos (trabalho, pessoal, urgente)
 - **Compare:** Compare dois contextos lado a lado
-

7. Context Buffers ☐

Definição

Context Buffers são janelas de contexto especializadas para interação com IA, oferecendo buffers dedicados que mantêm histórico, contexto e estado de cada sessão de IA.

Filosofema

“A IA é um espelho; o buffer é a moldura que define o reflexo.”

Rationale: Interações com IA frequentemente perdem contexto entre sessões. Context Buffers criam um ambiente persistente e organizado para cada IA, mantendo histórico, arquivos de contexto e estado de conversação.

Implementação

```
;; ~/.emacs.d/lisp/context-buffers.el

(defun context-buffer-alist '()
  "Associação de IAs para buffers.")

(defun context-buffer-prefix "AI-Context-"
  "Prefixo para buffers de IA.")

(cl-defstruct ai-context
  name                ; Nome da IA
  buffer              ; Buffer principal
  history-buffer      ; Buffer de histórico
  context-file        ; Arquivo de contexto
  provider            ; Provedor (openai, anthropic, local)
  endpoint            ; Endpoint da API
  system-prompt       ; Prompt de sistema
  session-id          ; ID da sessão)

(defun context-buffer-create (name &optional provider)
  "Cria um novo buffer de contexto para IA NAME."
  (interactive "sNome da IA: ")
  (let* ((buf-name (format "%s%s" context-buffer-prefix name))
         (hist-name (format "%s%s-HISTORY*" context-buffer-prefix name))
         (ctx-file (expand-file-name (format ".context-%s.json" name)
                                     user-emacs-directory)))
    (ctx (make-ai-context
          :name name
          :buffer buf-name
          :history-buffer hist-name
          :context-file ctx-file
          :provider (or provider (context-buffer-detect-provider)))))
```



```

                :endpoint (context-buffer-get-endpoint provider)
                :system-prompt (context-buffer-default-prompt name)
                :session-id (uuid-string)))
(push (cons name ctx) context-buffer-alist)
(context-buffer-setup-window ctx)
(message "□ Buffer de contexto '%s' criado!" name)
ctx))

(defun context-buffer-setup-window (ctx)
  "Configura layout de janelas para CTX."
  (delete-other-windows)
  (split-window-right)
  (switch-to-buffer (ai-context-buffer ctx))
  (other-window 1)
  (switch-to-buffer (ai-context-history-buffer ctx))
  (context-buffer-mode (ai-context-buffer ctx)))

(defun context-buffer-mode (buf)
  "Modo principal para buffers de IA."
  (with-current-buffer buf
    (use-local-map (copy-keymap text-mode-map))
    (local-set-key (kbd "C-c C-c") #'context-buffer-send)
    (local-set-key (kbd "C-c C-r") #'context-buffer-resend)
    (local-set-key (kbd "C-c C-k") #'context-buffer-clear)
    (insert (format "□ %s - Contexto de IA\n\n"
                    (ai-context-name buf)))
    (insert "Sistema: ")
    (insert (propertize (ai-context-system-prompt buf)
                        'face 'font-lock-comment-face))
    (insert "\n\n===== \n\n"))))

(defun context-buffer-send ()
  "Envia conteúdo para a IA."
  (interactive)
  (let* ((ctx (or (cl-find (buffer-name (current-buffer))
                           context-buffer-alist
                           :key #'cdr :test #'string-match)
                  (error "Não há contexto de IA neste buffer"))
         (prompt (context-buffer-get-prompt ctx))
         (history (context-buffer-get-history ctx)))
    (context-buffer-add-to-history ctx "user" prompt)
    (message "□ Enviando para %s..." (ai-context-provider ctx))
    (context-buffer-call-api ctx prompt
                             (lambda (response)
                               (context-buffer-display-response ctx response)))))

(defun context-buffer-call-api (ctx prompt callback)
  "Chama API da IA com PROMPT e executa CALLBACK."

```

```

(let* ((provider (ai-context-provider ctx))
      (json-payload (pcase provider
        ('openai
          (json-encode `((model . "gpt-4")
                        (messages . [((role . "system")
                                      (content . ,(ai-context-sys
                                                    (role . "user")
                                                    (content . ,prompt)))]))))
        ('anthropic
          (json-encode `((model . "claude-3-sonnet")
                        (system . ,(ai-context-system-prompt ctx)
                        (messages . [((role . "user")
                                      (content . ,prompt))])))))
      (url-retrieve
        (ai-context-endpoint ctx)
        (lambda (status)
          (goto-char (point-min))
          (re-search-forward "^$")
          (let ((response (buffer-substring (point) (point-max))))
            (funcall callback response)))
        ` (headers . (("Content-Type" . "application/json")
                     ("Authorization" . "Bearer ${OPENAI_API_KEY}")))
      t)))

(defun context-buffer-display-response (ctx response)
  "Mostra RESPONSE da IA no buffer."
  (with-current-buffer (ai-context-buffer ctx)
    (goto-char (point-max))
    (insert "\n Resposta:\n\n")
    (insert response)
    (insert "\n\n—————\n\n"))
  (context-buffer-add-to-history ctx "assistant" response))

;; Comandos
(global-set-key (kbd "C-c M-c c") #'context-buffer-create)
(global-set-key (kbd "C-c M-c l") (lambda ()
                                     (interactive)
                                     (dolist (pair context-buffer-alist)
                                       (message "%s" (car pair)))))

(defun context-buffer-default-prompt (name)
  "Retorna prompt de sistema padrão para NAME."
  (format "Você é um assistente de programação especializado.
  Suas respostas devem ser claras, concisas e técnicas.
  Responda em português quando apropriado.))

```

ELI5

É como ter várias janelas de chat abertas, mas cada uma é especializada. Uma janela é só para Python, outra para Emacs Lisp, outra para revisar código. Cada janela sabe do que você está falando e não mistura as conversas.

Recursos

- **Multi-provider:** Suporte a OpenAI, Anthropic, Ollama local
 - **Persistência:** Histórico salvo entre sessões
 - **Contexto de projeto:** Pode ler arquivos do projeto atual
 - **Voice input:** Suporte a entrada de voz (fala → texto)
 - **Code execution:** Pode executar código e mostrar resultados
-

□ Instalação e Integração

Carregamento Modular

Adicione ao seu `init.el`:

```
;; ~/.emacs.d/init.el

;; Future Improvements - Carregar na ordem de dependência
(add-to-list 'load-path (expand-file-name "lisp" user-emacs-directory))

;; 1. Corporal Mode (requer sistema háptico)
;; (require 'corporal)

;; 2. Context Buffers (base para NNC)
(require 'context-buffers)

;; 3. Context Management (gerencia contexts)
(require 'context-mgmt)

;; 4. NNC (usa context buffers)
;; (require 'nnc)

;; 5. Memory Graph (análise estática)
;; (require 'memory-graph)

;; 6. Context Navigation (navega contextos)
(require 'context-nav)

;; 7. Temporal Navigation (viaja no tempo)
;; (require 'temporal-nav)
```

Menu which-key

```
(with-eval-after-load 'which-key
  (defvar which-key-humanitarium-map (make-sparse-keymap))
  (define-key global-map (kbd "C-c ;") 'which-key-humanitarium-map)
  (which-key-add-key-based-replacements "C-c ;"
    ("c" . "corporal")
    ("n" . "nnc")
    ("m" . "memory-graph")
    ("t" . "temporal")
    ("j" . "context-nav")
    ("s" . "context-save")
    ("r" . "context-restore")
    ("b" . "context-buffer"))
  (which-key-add-keymap-based-replacements 'which-key-humanitarium-map
    "c" '(:keymap corporal-mode-map :which-key "corporal")
    "n" '(:keymap nnc-mode-map :which-key "nnc")
    "m" '(:keymap memory-graph-map :which-key "memory-graph")
    "t" '(:keymap temporal-nav-map :which-key "temporal")))
```

☐ Checklist de Implementação

Módulo	Status	Dependências	Complexidade
Corporal Mode ☐	☐	Dispositivo háptico	☐☐☐
NNC ☐	☐	llama.cpp, company-mode	☐☐☐☐
Memory Graph ☐	☐	graphviz, treesit	☐☐☐
Temporal Navigation ☐	☐	magit, project	☐☐
Context Navigation ☐	☐	Nenhuma	☐☐
Context Management ☐	☐	Nenhuma	☐☐☐
Context Buffers ☐	☐	context-mgmt	☐☐☐☐

☐ Visão de Futuro

“Emacs não é apenas um editor; é um universo de possibilidades.”

Estes módulos representam a evolução do EmacsIDE para um ambiente de desenvolvimento verdadeiramente inteligente e corporal. Cada módulo é independente, mas juntos formam um ecossistema coeso que transforma a experiência de programar.

Roadmap

1. **Fase 1** (Mês 1-2): Context Navigation + Context Management
2. **Fase 2** (Mês 2-3): Context Buffers + Corporal Mode

3. **Fase 3** (Mês 3-4): NNC + Memory Graph
 4. **Fase 4** (Mês 4-5): Temporal Navigation + Integração total
 5. **Fase 5** (Mês 5+): IA agents autônomos + Voice interface
-

Documento gerado em: 2026-04-16

Versão: 1.0.0

Emacs IDE - Beyond the Horizon □